

Wave 1 Candidate Registry

Denai Clinical Platform — Safe Modularization Governance

Classification: Extraction Governance Intelligence — Deterministic Preservation **Depends on:** Synchronization Analysis, Render Trigger Chain Analysis, Architecture Risk Consolidation, DOM Lifecycle Audit, Wave 1 Execution Playbook

Registry Structure

This registry is organized into four sections:

- 1. **Approved Wave 1 Candidates** — verified or probably-pure targets eligible for extraction
 - 2. **Conditional Candidates** — targets requiring investigation before eligibility can be confirmed
 - 3. **Deferred Wave 2+ Candidates** — targets not eligible for Wave 1 but potentially eligible in future waves
 - 4. **Permanently Forbidden Targets** — targets that must never be extracted as-is under any wave
-

Section 1: Approved Wave 1 Candidates

CANDIDATE-001: BRAND

Field	Value
Candidate name	BRAND
Current location/context	Module-scope constant object in main application file
Category	Constants

Purity confidence	VERIFIED PURE
Reasons for classification	Named constant object. No DOM access, no state read/write, no timer registration, no render calls. Pure data.
Direct dependencies	None (self-contained constant)
Reverse dependencies (major callers)	Label construction functions, badge text generation functions, any consumer referencing brand strings
Hidden coupling concerns	None identified. Does not intersect with S, tempState, or any render path.
Synchronization concerns	None. Static read-only object.
Render/orchestration adjacency	None.
Extraction tier	Tier 1
Human review required	N
Claude Opus escalation required	N
Recommended extraction order	1st within Tier 1 (no dependencies)
Safe-to-extract status	SAFE NOW

CANDIDATE-002: PRESETS

Field	Value
Candidate name	PRESETS
Current location/context	Module-scope constant object in main application file
Category	Constants

Purity confidence	VERIFIED PURE
Reasons for classification	Explicitly listed as an allowed extraction target in Playbook Section 2.3. Static constant object containing preset configurations.
Direct dependencies	None (self-contained constant)
Reverse dependencies (major callers)	<code>applyPreset()</code> (forbidden function — reads PRESETS but is not extracted); any UI that renders preset labels
Hidden coupling concerns	<code>applyPreset()</code> reads from PRESETS and calls <code>Object.assign(tempState, ...)</code> . Extracting PRESETS does not disturb this — it remains a passive read target. The call site in <code>applyPreset()</code> receives PRESETS as an import; <code>applyPreset()</code> itself remains in main application.
Synchronization concerns	None. <code>applyPreset()</code> is not extracted; the PRESETS constant is not involved in any timer or observer.
Render/orchestration adjacency	Adjacent to <code>applyPreset()</code> caller context only. Extraction does not touch <code>applyPreset()</code> .
Extraction tier	Tier 1
Human review required	N
Claude Opus escalation required	N
Recommended extraction order	2nd within Tier 1
Safe-to-extract status	SAFE NOW

CANDIDATE-003: `MULTI_TO_SINGLE` (treatment mapping object)

Field	Value
Candidate name	<code>MULTI_TO_SINGLE</code>
Current	Module-scope constant object in main application file

location/context	
Category	Lookup table
Purity confidence	VERIFIED PURE
Reasons for classification	Explicitly listed as an allowed extraction target in Playbook Section 2.3. Pure mapping object with no side effects.
Direct dependencies	None (self-contained lookup table)
Reverse dependencies (major callers)	Treatment selection logic, multi-tooth mode treatment normalization paths. Callers remain in main application.
Hidden coupling concerns	Must verify at pre-extraction that no call site passes this object into a function that mutates S. The object itself is pure; the call sites must not create an implicit dependency on mutation ordering.
Synchronization concerns	None. Static mapping.
Render/orchestration adjacency	None directly. Callers may be inside render-adjacent functions but MULTI_TO_SINGLE itself is a lookup table only.
Extraction tier	Tier 1
Human review required	N
Claude Opus escalation required	N
Recommended extraction order	3rd within Tier 1
Safe-to-extract status	SAFE NOW

CANDIDATE-004: Treatment label maps

Field	Value
	Treatment label maps (all pure label/display name mapping objects)

Candidate name	for treatment types)
Current location/context	Module-scope constant objects in main application file
Category	Lookup table
Purity confidence	VERIFIED PURE
Reasons for classification	Pure string-to-string or key-to-label mappings. No state access, no DOM access, no render calls. Explicitly in allowed category (Playbook Section 2.3).
Direct dependencies	None beyond native JavaScript
Reverse dependencies (major callers)	Label construction functions, AI card content builders, comparison table renderers. All callers remain in main application.
Hidden coupling concerns	Verify that no label map contains entries that reference transient UI flags (<code>editing</code> , <code>whyOpen</code> , <code>historyOpen</code>). If any map is keyed by a transient flag name, it falls under the forbidden DEFAULT_STATE transient flag rule.
Synchronization concerns	None.
Render/orchestration adjacency	Consumed by render-path functions, but the maps themselves carry no render responsibilities.
Extraction tier	Tier 1
Human review required	N
Claude Opus escalation required	N
Recommended extraction order	4th within Tier 1
Safe-to-extract status	SAFE NOW

CANDIDATE-005: Tooth classification maps

Field	Value
Candidate name	Tooth classification maps (tooth type → classification mappings; arch maps; quadrant maps)
Current location/context	Module-scope constant objects in main application file
Category	Lookup table / Classification helper
Purity confidence	VERIFIED PURE
Reasons for classification	Pure classification mapping objects. No state reads, no DOM writes. Explicitly listed in Playbook Section 2.3 (tooth type classification, condition category mapping).
Direct dependencies	None beyond native JavaScript
Reverse dependencies (major callers)	<code>ClinicalEngine.process()</code> , <code>ClinicalEngine.processCompound()</code> , pure classification functions. Callers remain in main application or are themselves Wave 1 Tier 4 targets.
Hidden coupling concerns	If any tooth classification map is used inside <code>updateCrownCardState()</code> (which reads <code>DOM.classList</code>), that usage is at the call site — the map itself remains pure. Verify that no map entry is derived from DOM state or S at construction time.
Synchronization concerns	None.
Render/orchestration adjacency	None on the maps themselves.
Extraction tier	Tier 1
Human review required	N
Claude Opus escalation required	N
Recommended extraction order	5th within Tier 1

Safe-to-extract status	SAFE NOW
-------------------------------	----------

CANDIDATE-006: Condition category maps

Field	Value
Candidate name	Condition category maps (condition → category classification mappings)
Current location/context	Module-scope constant objects in main application file
Category	Lookup table / Classification helper
Purity confidence	VERIFIED PURE
Reasons for classification	Pure lookup/classification objects. No state access, no DOM interaction, no timers. Listed in Playbook Section 2.3.
Direct dependencies	None beyond native JavaScript
Reverse dependencies (major callers)	<code>ClinicalEngine.process()</code> , <code>ClinicalEngine.processCompound()</code> , any classification helper functions.
Hidden coupling concerns	Verify that condition keys do not map to values that include the RESTORATIVE/MISSING_SINGLE/MISSING_MULTI branch names as executable logic rather than pure labels. The routing logic that consumes these keys stays in <code>ClinicalEngine.process()</code> (frozen per Boundary B-09).
Synchronization concerns	None.
Render/orchestration adjacency	None.
Extraction tier	Tier 1
Human review required	N
Claude Opus	

escalation required	N
Recommended extraction order	6th within Tier 1 (same tier as tooth classification maps; may be extracted in parallel)
Safe-to-extract status	SAFE NOW

CANDIDATE-007: `DEFAULT_STATE` clinical field defaults (partial extraction)

Field	Value
Candidate name	<code>DEFAULT_STATE</code> — clinical field defaults only (excluding transient UI flags)
Current location/context	Module-scope constant/object in main application file
Category	Constants
Purity confidence	PROBABLY PURE
Reasons for classification	The clinical field portion is a pure data object. However, the full <code>DEFAULT_STATE</code> object contains transient UI flags (<code>editing</code> , <code>whyOpen</code> , <code>historyOpen</code>) that are explicitly forbidden from extraction due to the destructure-blocklist serialization pattern (Playbook Section 3.4, Rule 11.9). Only the clinical-field portion is eligible.
Direct dependencies	None beyond native JavaScript for the clinical fields subset
Reverse dependencies (major callers)	<code>setState()</code> (forbidden — stays in main application); <code>switchPatient()</code> (forbidden); any code path that resets clinical defaults.
Hidden coupling concerns	Critical: The extracted partial must not include <code>editing</code> , <code>whyOpen</code> , or <code>historyOpen</code> . These three flags must remain in-place in the main application file to preserve the <code>const { editing, whyOpen, historyOpen, ...serializable } = S</code> destructure-blocklist pattern in <code>saveState()</code> . Any extraction that changes which field names are present in the main-file <code>DEFAULT_STATE</code> risks altering persistence behavior. The extracted clinical fields must be verified not to be used in the blocklist

	destructure path.
Synchronization concerns	The 200ms <code>saveState()</code> debounce and its <code>renderPatientList()</code> side-effect chain must remain untouched. DEFAULT_STATE extraction must not alter or move any code adjacent to the <code>saveState()</code> timer.
Render/orchestration adjacency	Adjacent to <code>saveState()</code> persistence path. The clinical defaults themselves carry no render responsibilities.
Extraction tier	Tier 1
Human review required	Y — Playbook Section 14, H-02: human review required to confirm the transient flag exclusion is correct and complete.
Claude Opus escalation required	N (unless the boundary between clinical fields and transient flags is ambiguous in the actual code — if ambiguous, escalate per E-01)
Recommended extraction order	Last in Tier 1, after all other constant extractions, to allow human review of flag boundary.
Safe-to-extract status	SAFE AFTER VALIDATION

CANDIDATE-008: Pure recommendation label maps and lookup tables

Field	Value
Candidate name	Pure recommendation label maps (recommendation category → display label; risk label maps; classification label maps)
Current location/context	Module-scope constant objects in main application file
Category	Lookup table
Purity confidence	VERIFIED PURE
Reasons for classification	Pure string lookup tables. No state access, no DOM writes. Explicitly listed as allowed in Playbook Section 2.3.
Direct dependencies	None beyond native JavaScript
	<code>ClinicalEngine.process()</code> ,

Reverse dependencies (major callers)	<code>ClinicalEngine.processCompound()</code> , <code>calcAI()</code> , <code>calcAIMulti()</code> , pure classification helpers. All callers remain in main application or are Tier 4 targets.
Hidden coupling concerns	Note: These maps may be consumed by both <code>calcAI()</code> and <code>ClinicalEngine.process()</code> . Per documented divergence (Playbook Section 5.3, Rule 11.8), these two paths produce semantically different recommendation objects for restorative conditions. The label maps themselves do not cause or resolve this divergence — they are passive lookup targets. Verify at pre-extraction that no map entry encodes branching logic that would affect the <code>calcAI</code> vs. <code>ClinicalEngine</code> semantic difference.
Synchronization concerns	None on the maps.
Render/orchestration adjacency	Consumed by render-adjacent functions via <code>ClinicalEngine</code> , but the maps carry no render responsibilities.
Extraction tier	Tier 1
Human review required	N
Claude Opus escalation required	N
Recommended extraction order	Within Tier 1, after core constant objects. May be extracted in parallel with tooth/condition classification maps.
Safe-to-extract status	SAFE NOW

CANDIDATE-009: Pure string formatting helpers — name formatting

Field	Value
Candidate name	Name formatting functions (patient name display formatting, label case normalization)
Current location/context	Function declarations in main application file

Category	Formatter
Purity confidence	VERIFIED PURE
Reasons for classification	Accept string inputs, return formatted string outputs. No state reads, no DOM writes, no timers. Explicitly listed as allowed in Playbook Section 2.3 (pure string formatting functions).
Direct dependencies	None beyond native JavaScript string methods
Reverse dependencies (major callers)	<code>renderPatientList()</code> (forbidden — stays in main application), <code>renderPatientDisplay()</code> (forbidden — stays in main application), AI card content builders. All callers remain in main application.
Hidden coupling concerns	Verify that name formatting functions do not read from any module-scoped variable for locale/format settings. If format preference is read from S, the function is not pure.
Synchronization concerns	None.
Render/orchestration adjacency	Called from within render functions, but the formatting functions themselves carry no render responsibilities.
Extraction tier	Tier 2
Human review required	N
Claude Opus escalation required	N
Recommended extraction order	1st within Tier 2
Safe-to-extract status	SAFE NOW

CANDIDATE-010: Pure string formatting helpers — label construction and badge text

Field	Value

Candidate name	Label construction functions, badge text generation functions
Current location/context	Function declarations in main application file
Category	Formatter
Purity confidence	PROBABLY PURE
Reasons for classification	Produce display label strings from input parameters. Likely pure. Explicitly listed as allowed category in Playbook Section 2.3.
Direct dependencies	May depend on Tier 1 label maps (CANDIDATE-008). All dependencies must be confirmed as Wave 1 extracted modules or native JavaScript before extraction.
Reverse dependencies (major callers)	AI card content builders (<code>buildAICardStructure</code> , <code>updateAICard</code> — both forbidden, stay in main application), comparison table renderers (forbidden), treatment card renderers (forbidden).
Hidden coupling concerns	Verify that badge text generation does not read <code>S.tx</code> , <code>S.condition</code> , or any state field directly from module scope. If inputs are always passed as parameters, the function is pure. Any direct state read = FAIL.
Synchronization concerns	None if pure.
Render/orchestration adjacency	Called from within forbidden render functions. Extraction does not move those render functions — only the formatting helper is extracted.
Extraction tier	Tier 2
Human review required	N
Claude Opus escalation required	N
Recommended extraction order	2nd within Tier 2, after name formatting helpers
Safe-to-extract status	SAFE AFTER VALIDATION

CANDIDATE-011: Pure string formatting helpers — date formatting

Field	Value
Candidate name	Date formatting functions (clinical date display, history timestamp formatting)
Current location/context	Function declarations in main application file
Category	Formatter
Purity confidence	PROBABLY PURE
Reasons for classification	Accept date inputs, return formatted string outputs. Explicitly in allowed category (Playbook Section 2.3).
Direct dependencies	Native JavaScript Date methods only (expected)
Reverse dependencies (major callers)	<code>History.render()</code> (forbidden — stays in main application), <code>renderPatientList()</code> (forbidden).
Hidden coupling concerns	Verify no implicit locale read from <code>window.navigator</code> or <code>S</code> . Any browser API access = FAIL (DOM API scan).
Synchronization concerns	None.
Render/orchestration adjacency	Called from History and sidebar render paths. Formatter itself carries no render responsibility.
Extraction tier	Tier 2
Human review required	N
Claude Opus escalation required	N
Recommended extraction order	3rd within Tier 2
Safe-to-extract status	SAFE AFTER VALIDATION

CANDIDATE-012: Pure number formatting helpers — score display formatting

Field	Value
Candidate name	Score display formatting functions (clinical score → display string, score percentage formatting)
Current location/context	Function declarations in main application file
Category	Formatter
Purity confidence	PROBABLY PURE
Reasons for classification	Accept numeric inputs, return formatted strings. Explicitly listed as allowed in Playbook Section 2.3.
Direct dependencies	Native JavaScript number methods only (expected)
Reverse dependencies (major callers)	<code>updateAICard()</code> (forbidden), <code>renderCost()</code> (forbidden), <code>renderGraph()</code> (forbidden). All callers stay in main application.
Hidden coupling concerns	Verify that score formatters do not read scoring weights or normalization constants directly from module scope. If scoring constants are needed, they should be passed as parameters or come from a Tier 1 extracted constant.
Synchronization concerns	None.
Render/orchestration adjacency	Adjacent to AI card and graph render paths, but the formatters themselves are pure.
Extraction tier	Tier 2
Human review required	N
Claude Opus escalation required	N
Recommended extraction order	4th within Tier 2 (may be extracted in parallel with date formatting)

Safe-to-extract status	SAFE AFTER VALIDATION
------------------------	-----------------------

CANDIDATE-013: Pure number formatting helpers — cost display formatting

Field	Value
Candidate name	Cost display formatting functions (numeric cost → formatted currency string)
Current location/context	Function declarations in main application file
Category	Formatter
Purity confidence	PROBABLY PURE
Reasons for classification	Accept numeric cost values, return display strings. Explicitly listed as allowed in Playbook Section 2.3. Distinct from <code>computeCosts()</code> which is a Tier 4 calculation helper.
Direct dependencies	Native JavaScript number/locale methods only (expected). Must NOT depend on <code>computeCosts()</code> being called first — formatting is input-agnostic.
Reverse dependencies (major callers)	<code>renderCost()</code> (forbidden).
Hidden coupling concerns	Verify that cost display formatters do not read any currency/locale setting from <code>s</code> or from <code>window</code> . If they use <code>Intl.NumberFormat</code> with a hardcoded locale, that is acceptable. If locale is read from state or browser, the Browser API scan applies.
Synchronization concerns	None.
Render/orchestration adjacency	Called from <code>renderCost()</code> which is forbidden. Formatter itself is pure.
Extraction tier	Tier 2
Human review required	N

Claude Opus escalation required	N
Recommended extraction order	5th within Tier 2
Safe-to-extract status	SAFE AFTER VALIDATION

CANDIDATE-014: Pure validation functions — field-level range and type checks

Field	Value
Candidate name	Standalone field-level validation functions (range checks, allowed-value sets, type checks for individual clinical fields)
Current location/context	Function declarations in main application file, potentially also within or adjacent to <code>StateValidator</code>
Category	Validator
Purity confidence	PROBABLY PURE
Reasons for classification	Accept a value, return a boolean or validation result. No state mutation, no render calls. Explicitly listed as allowed in Playbook Section 2.3.
Direct dependencies	May depend on Tier 1 constant objects for allowed-value sets. Must be confirmed before extraction.
Reverse dependencies (major callers)	<code>StateValidator.validate()</code> (Tier 3 conditional target — see CANDIDATE-016), <code>setState()</code> (forbidden), any UI that displays inline validation feedback.
Hidden coupling concerns	These functions may be called from within <code>StateValidator</code> . If they are extracted before <code>StateValidator</code> (which is Tier 3), they become dependencies of the Tier 3 extraction. This is the correct extraction order and is safe. Verify that no field-level validator pushes to a <code>StateDiffs</code> array or emits an event (per Playbook Section 5.8 and E-04).
Synchronization	

concerns	None if pure.
Render/orchestration adjacency	None directly. <code>setState()</code> calls these validators before committing state, but the validators themselves do not touch the render path.
Extraction tier	Tier 3
Human review required	N
Claude Opus escalation required	N (unless a validator is discovered to push to StateDiffs — then escalate per E-04)
Recommended extraction order	1st within Tier 3, before <code>StateValidator.validate()</code>
Safe-to-extract status	SAFE AFTER VALIDATION

CANDIDATE-015: Pure score normalization functions

Field	Value
Candidate name	Score normalization functions (raw clinical score → normalized score in bounded range)
Current location/context	Within or adjacent to <code>ClinicalEngine</code> in main application file
Category	Pure calculation helper
Purity confidence	PROBABLY PURE
Reasons for classification	Accept numeric inputs and scoring parameters, return normalized numeric output. Explicitly listed as allowed in Playbook Section 2.3 and Tier 4 of the extraction sequence. These are to be extracted as standalone functions, NOT as a refactored <code>ClinicalEngine</code> module.
Direct dependencies	Likely scoring weight constants (Tier 1 or inline constants). Must be confirmed.
Reverse	<code>ClinicalEngine.process()</code> (stays in main application),

dependencies (major callers)	<code>ClinicalEngine.processCompound()</code> (stays in main application), potentially <code>calcAI()</code> and <code>calcAIMulti()</code> (both forbidden for extraction — see Section 3).
Hidden coupling concerns	Critical: Per Playbook Section 5.3 and Rule 11.8, <code>calcAI()</code> and <code>ClinicalEngine.process()</code> produce different recommendation objects for restorative conditions. If any score normalization function is called from both paths and its output feeds into the diverging recommendation structures, extracting it as a shared function must not alter either path's output. The behavioral equivalence baseline (Section 4.2) must include outputs from both caller contexts. Any difference post-extraction = RF-05 red flag.
Synchronization concerns	Verify the 160ms material window (T01) and typewriter timer (T05) are unaffected after Tier 4 extraction (Playbook Section 8.4).
Render/orchestration adjacency	Called from within <code>ClinicalEngine</code> which feeds <code>renderMainPanels</code> . The normalization function itself has no render responsibilities.
Extraction tier	Tier 4
Human review required	Y — Playbook Section 14, H-01: clinical calculation logic requires human clinical engineering review.
Claude Opus escalation required	N unless the normalization function's boundary within <code>ClinicalEngine</code> is ambiguous (escalate per E-02 if so)
Recommended extraction order	1st within Tier 4 (lowest coupling risk among clinical helpers)
Safe-to-extract status	SAFE AFTER VALIDATION

CANDIDATE-016: Pure risk classification functions

Field	Value
Candidate name	Risk classification functions (clinical parameter set → risk category classification)
Current location/context	Within <code>ClinicalEngine</code> in main application file
Category	Classification helper / Pure calculation helper

Purity confidence	PROBABLY PURE
Reasons for classification	Accept clinical parameters as arguments, return a risk classification. No S access, no DOM writes. Listed as allowed in Playbook Section 2.3 (pure classification functions). To be extracted as standalone, not as refactored ClinicalEngine module.
Direct dependencies	Risk scoring weight constants (Tier 1), score normalization functions (CANDIDATE-015 — must be extracted first if there is a dependency).
Reverse dependencies (major callers)	<code>ClinicalEngine.process()</code> , <code>ClinicalEngine.processCompound()</code> , <code>renderRisk()</code> (forbidden — consumes result, not the function).
Hidden coupling concerns	Must verify against Playbook Section 5.2: does this function assume it is called only from a specific branch of <code>ClinicalEngine.process()</code> (MISSING_SINGLE, MISSING_MULTI, or RESTORATIVE)? If it carries an implicit precondition from the routing branch, it may require the precondition to be expressed as an explicit parameter before extraction. If the boundary is ambiguous, escalate per E-02.
Synchronization concerns	Same as CANDIDATE-015: verify timing windows after Tier 4 extraction.
Render/orchestration adjacency	Output consumed by <code>renderRisk()</code> (forbidden), but the classification function itself has no render responsibility.
Extraction tier	Tier 4
Human review required	Y — H-01 applies.
Claude Opus escalation required	Conditional — escalate per E-02 if the routing branch boundary is ambiguous
Recommended extraction order	2nd within Tier 4, after score normalization functions
Safe-to-extract status	SAFE AFTER VALIDATION

CANDIDATE-017: Pure classification functions — tooth type classification

Field	Value
Candidate name	Tooth type classification function(s) (tooth number → tooth type category)
Current location/context	Function declarations in main application file, possibly adjacent to tooth classification maps
Category	Classification helper
Purity confidence	PROBABLY PURE
Reasons for classification	Accept tooth identifier as input, return tooth type classification. Uses tooth classification maps (CANDIDATE-005). Explicitly listed as allowed in Playbook Section 2.3.
Direct dependencies	Tooth classification maps (CANDIDATE-005 — must be extracted in Tier 1 first)
Reverse dependencies (major callers)	<code>ClinicalEngine.process()</code> , <code>ClinicalEngine.processCompound()</code> , <code>computeCosts()</code> (CANDIDATE-019), <code>getCrownMaterial()</code> (CANDIDATE-020).
Hidden coupling concerns	Verify against Section 5.4: does the classification function read any DOM state (e.g., a <code>.classList</code> written by a prior render)? If it reads tooth type from DOM rather than from its parameter, it is FORBIDDEN.
Synchronization concerns	None if pure.
Render/orchestration adjacency	None on the function itself.
Extraction tier	Tier 4
Human review required	Y — H-01 applies (clinical calculation logic).
Claude Opus escalation required	N
Recommended extraction order	3rd within Tier 4

Safe-to-extract status	SAFE AFTER VALIDATION
-------------------------------	-----------------------

CANDIDATE-018: Pure recommendation mapping and classification functions

Field	Value
Candidate name	Pure recommendation mapping functions (clinical result → recommendation label/category)
Current location/context	Within or adjacent to <code>ClinicalEngine</code> in main application file
Category	Classification helper / Pure calculation helper
Purity confidence	PROBABLY PURE
Reasons for classification	Accept clinical output parameters, return recommendation classification or label. No state mutation, no DOM access. Explicitly listed as allowed in Playbook Section 2.3.
Direct dependencies	Recommendation label maps (CANDIDATE-008 — Tier 1), condition category maps (CANDIDATE-006 — Tier 1).
Reverse dependencies (major callers)	<code>ClinicalEngine.process()</code> (stays in main application), potentially <code>calcAI()</code> and <code>calcAIMulti()</code> (both forbidden for extraction).
Hidden coupling concerns	Critical — calcAI vs. ClinicalEngine divergence (Playbook Section 5.3, Rule 11.8): If a recommendation mapping function is called from both <code>calcAI()</code> and <code>ClinicalEngine.process()</code> , and the two paths produce semantically different recommendation objects, the extracted function must not alter the output of either path. The pre-extraction behavioral baseline must record outputs from both caller contexts independently. Any post-extraction divergence = RF-05.
Synchronization concerns	None if pure.
Render/orchestration adjacency	Output consumed by AI card render functions (forbidden). Function itself has no render responsibility.

Extraction tier	Tier 4
Human review required	Y — H-01 applies.
Claude Opus escalation required	Conditional — escalate per E-03 if the function's role in the calcAI vs. ClinicalEngine divergence cannot be fully traced
Recommended extraction order	4th within Tier 4, after classification helpers
Safe-to-extract status	SAFE AFTER VALIDATION

Section 2: Conditional Candidates Requiring Investigation

CANDIDATE-019: `computeCosts(state, ai)`

Field	Value
Candidate name	<code>computeCosts(state, ai)</code>
Current location/context	Function declaration in main application file
Category	Pure calculation helper (conditional)
Purity confidence	REQUIRES INVESTIGATION
Reasons for classification	Explicitly listed as eligible only if verified to be pure (Playbook Section 2.3 and Section 4). Function accepts state and ai as parameters which is consistent with pure pattern (receives state as parameter, does not import it). However, purity is not confirmed — must be investigated.
Direct dependencies	Likely: tooth classification functions (CANDIDATE-017), cost constants (potentially Tier 1), possibly treatment label maps (CANDIDATE-004). Full dependency chain requires depth-2 scan (Playbook Section 18.2).
Reverse dependencies (major)	<code>renderCost()</code> (forbidden — stays in main application), potentially

callers)	<code>generateReport()</code> (forbidden).
Hidden coupling concerns	Must verify: (1) does the function read any field from <code>state</code> that is a transient UI flag (<code>editing</code> , <code>whyOpen</code> , <code>historyOpen</code>)? (2) does it call any render function? (3) does it access <code>localStorage</code> to read cost data? (4) does it mutate the <code>ai</code> object passed in (e.g., setting <code>ai._previewScore</code> as documented in Section 5.9 and Rule 11.8)? If <code>computeCosts()</code> sets <code>ai._previewScore</code> , it is NOT pure — it mutates its input parameter.
Synchronization concerns	If called from within <code>updatePreview()</code> path, the <code>tempState</code> is passed as <code>state</code> . Verify that the extraction does not alter the 160ms material window or the preview timer cancellation behavior.
Render/orchestration adjacency	Called from within <code>renderCost()</code> (forbidden). Must not be placed in any render path file.
Extraction tier	Tier 4 (conditional — only if investigation confirms purity)
Human review required	Y — H-01 applies.
Claude Opus escalation required	Conditional — if the function's interaction with the <code>ai</code> object is ambiguous (especially regarding <code>ai._previewScore</code> mutation), escalate per E-01
Recommended extraction order	After CANDIDATE-017 (tooth type classification). Last in Tier 4 before conditional extractions.
Safe-to-extract status	DEFER TO WAVE 2+ unless investigation confirms purity

Required pre-investigation checklist:

- S-read scan on full function body
- Check for `ai._previewScore` write (Section 5.9)
- Check for any mutation of the `ai` or `state` parameter objects
- Check for `localStorage` access
- Depth-2 dependency scan on all called sub-functions

CANDIDATE-020: `getCrownMaterial(state)`

Field	Value
Candidate name	<code>getCrownMaterial(state)</code>
Current location/context	Function declaration in main application file
Category	Pure calculation helper (conditional)
Purity confidence	REQUIRES INVESTIGATION
Reasons for classification	Explicitly listed as eligible only if verified to be pure (Playbook Section 2.3 and Section 4). Accepts state as a parameter which is consistent with pure pattern. Purity not confirmed.
Direct dependencies	Tooth classification functions (CANDIDATE-017), tooth classification maps (CANDIDATE-005), possibly condition category maps (CANDIDATE-006). Full dependency scan required.
Reverse dependencies (major callers)	<code>renderMaterial()</code> (forbidden — contains <code>_matFadeTimer</code> registration; stays in main application). Called from the 160ms timer callback inside <code>renderMaterial()</code> .
Hidden coupling concerns	Must verify: (1) does <code>getCrownMaterial()</code> read any field of <code>state</code> beyond its tooth/condition inputs? (2) does it reference <code>_matFadeTimer</code> or any module-scoped timer variable? (3) does it call any render function? (4) does it access DOM to read <code>.classList</code> state (Section 5.4 risk)? The DOM-read precondition check is critical here: if crown material selection depends on a class written by <code>renderTxCards()</code> , the function reads DOM state and is FORBIDDEN.
Synchronization concerns	Critical: This function is called from within the <code>_matFadeTimer</code> 160ms callback in <code>renderMaterial()</code> . The 160ms material window (T01) is a documented behavioral contract (Rule 11.6). The extraction must not alter the timing or the cancellation behavior of <code>_matFadeTimer</code> . <code>getCrownMaterial()</code> itself must be pure and synchronous — it must not register or cancel any timer. The timer remains in <code>renderMaterial()</code> in the main application.
Render/orchestration adjacency	Directly adjacent to <code>_matFadeTimer</code> . Extraction must not disturb the timer — only the pure calculation function is extracted, not the timer or the DOM write.

Extraction tier	Tier 4 (conditional — only if investigation confirms purity)
Human review required	Y — H-01 applies.
Claude Opus escalation required	Conditional — escalate per E-01 if DOM-read dependency is discovered mid-investigation
Recommended extraction order	After CANDIDATE-017 (tooth type classification). Concurrent with CANDIDATE-019 investigation; extraction must not proceed until <code>_matFadeTimer</code> timing baseline is confirmed.
Safe-to-extract status	DEFER TO WAVE 2+ unless investigation confirms purity

Required pre-investigation checklist:

- DOM API scan on full function body (especially `.classList` read — Section 5.4)
- S-read scan (must receive state only as parameter, not import module-scoped S)
- Timer scan (must not reference `_matFadeTimer`, `setTimeout`, or `clearTimeout`)
- Depth-2 dependency scan

CANDIDATE-021: `StateValidator.validate(field, value)`

Field	Value
Candidate name	<code>StateValidator.validate(field, value)</code> — validation logic only
Current location/context	<code>StateValidator</code> object in main application file
Category	Validator (conditional)
Purity confidence	REQUIRES INVESTIGATION
Reasons for classification	Listed as eligible in Playbook Section 2.3 if and only if it is a pure function that does not mutate S and does not trigger any render. The <code>StateValidator</code> object as a whole may have side channels (logging to a module-scoped array, emitting an event). Investigation required.

Direct dependencies	Field-level validation functions (CANDIDATE-014 — must be extracted in Tier 3 first if there is a dependency), possibly allowed-value constant sets from Tier 1.
Reverse dependencies (major callers)	<code>setState()</code> (forbidden — stays in main application). <code>setState()</code> is the primary gate that calls <code>StateValidator.validate()</code> . Any direct call sites outside <code>setState()</code> must be inventoried.
Hidden coupling concerns	Critical — per Playbook Section 5.8 (StateDiffs side-channel): If <code>StateValidator.validate()</code> anywhere in its call chain pushes to a <code>StateDiffs</code> array, the function is FORBIDDEN. The diff-tracking is tied to <code>setState()</code> orchestration and must not be extracted. Per Playbook E-04: If the validator has a side-channel write (logging, event emission, diff push), escalation to Claude Opus is required before re-scoping.
Synchronization concerns	<code>setState()</code> is the 200ms debounce trigger chain entry point. The validator must not carry any timer references.
Render/orchestration adjacency	Called from <code>setState()</code> which is adjacent to the entire render chain. The validator itself must not call any render function.
Extraction tier	Tier 3 (conditional — only if investigation confirms no side channels)
Human review required	Y — H-03 applies: human review required to confirm no side-channel behavior.
Claude Opus escalation required	Conditional — escalate per E-04 if side-channel write is discovered
Recommended extraction order	2nd within Tier 3, after standalone field-level validators
Safe-to-extract status	DEFER TO WAVE 2+ unless investigation confirms absence of side channels

Required pre-investigation checklist:

- Search for any `StateDiffs` push, array append, or event emission in full `StateValidator` body
- Search for any `setState()`, `saveState()`, or render call
- Search for any module-scoped variable read or write beyond the return value

- Confirm `validate(field, value)` method can run in isolation in pure Node.js with no mocks

CANDIDATE-022: `showToast()`

Field	Value
Candidate name	<code>showToast()</code>
Current location/context	Function declaration in main application file
Category	Conditional candidate
Purity confidence	REQUIRES INVESTIGATION
Reasons for classification	Explicitly listed as conditional in Playbook Section 3.4: eligible if it is a pure function that only constructs and appends a toast element without reading from S; forbidden if it reads from S or has any implicit state dependency.
Direct dependencies	DOM APIs (if it appends to <code>document.body</code> or a container element, it is NOT pure — DOM append is a side effect that fails the DOM API scan). Expected: the function writes to DOM; it may or may not be extractable depending on how it is structured.
Reverse dependencies (major callers)	<code>selectTx()</code> (forbidden), <code>applyAIRec()</code> (forbidden), <code>generateReport()</code> (forbidden), potentially other action handlers.
Hidden coupling concerns	If <code>showToast()</code> reads from <code>s</code> for any content (patient name, treatment label, toast message), it is FORBIDDEN. If it appends directly to <code>document.body</code> , it performs DOM mutation and fails the DOM API scan — however, whether a DOM-appending toast wrapper is considered “pure” under Playbook Section 2.3 requires human judgment. The Playbook states eligibility requires no DOM access; a toast that appends to DOM does access document.
Synchronization concerns	If <code>showToast()</code> uses any <code>setTimeout</code> for auto-dismiss, it registers a timer and is FORBIDDEN.
Render/orchestration adjacency	Called from state-mutation and render-trigger functions. The toast itself is a display side effect.

Extraction tier	Tier 5 (conditional — human review required before extraction attempt)
Human review required	Y — H-04 applies: human review required to confirm no implicit S dependency.
Claude Opus escalation required	N
Recommended extraction order	First and only Tier 5 target (if approved following investigation).
Safe-to-extract status	DEFER TO WAVE 2+ unless investigation confirms eligibility

Required pre-investigation checklist:

- S-read scan on full function body
- DOM API scan — if `document.` or `getElementById` appears: determine if this disqualifies it per Playbook Section 2.1, or if the toast wrapper pattern is treated as a pure constructor by the team
- Timer scan (any `setTimeout` for dismiss = FAIL)
- If all scans pass: present to human review before proceeding

Section 3: Deferred Wave 2+ Candidates

CANDIDATE-023: `ClinicalEngine.process()`

Field	Value
Candidate name	<code>ClinicalEngine.process()</code>
Current location/context	<code>ClinicalEngine</code> object in main application file
Category	Pure calculation helper (deferred)
Purity confidence	REQUIRES INVESTIGATION (Wave 2 scope)
	Explicitly forbidden for Wave 1 (Playbook Section 3.4). Contains the

Reasons for classification	MISSING_SINGLE, MISSING_MULTI, RESTORATIVE routing logic which is load-bearing and must not be separated from its callers prematurely. Eligible for Wave 2 only after its internal pure sub-functions are extracted individually in Wave 1 (Tier 4).
Direct dependencies	All Tier 4 pure sub-functions (CANDIDATE-015 through CANDIDATE-018). Full extraction of those sub-functions is a prerequisite.
Reverse dependencies (major callers)	<code>render(S)</code> (via orchestration chain), <code>generateReport()</code> .
Hidden coupling concerns	Routing branches (MISSING_SINGLE, MISSING_MULTI, RESTORATIVE) must remain intact. The documented semantic divergence from <code>calcAI()</code> must be preserved (Rule 11.8, Boundary B-09).
Synchronization concerns	Full render chain dependency.
Render/orchestration adjacency	Core orchestration input.
Extraction tier	DEFERRED — Wave 2
Human review required	Y
Claude Opus escalation required	Y — E-02 conditions likely apply
Recommended extraction order	Wave 2, after all Wave 1 Tier 4 extractions are validated
Safe-to-extract status	DEFER TO WAVE 2+

CANDIDATE-024: `ClinicalEngine.processCompound()`

Field	Value
Candidate name	<code>ClinicalEngine.processCompound()</code>

Current location/context	<code>ClinicalEngine</code> object in main application file
Category	Pure calculation helper (deferred)
Purity confidence	REQUIRES INVESTIGATION (Wave 2 scope)
Reasons for classification	Same prohibition as <code>ClinicalEngine.process()</code> (Playbook Section 3.4). Additionally, this function contains the two-pass SVG compound highlight dependency chain — its extraction must account for the frozen second-pass overwrite contract (Boundary B-08, Rule 11.1).
Direct dependencies	All Tier 4 pure sub-functions. <code>ClinicalEngine.process()</code> sub-functions.
Reverse dependencies (major callers)	<code>render(S)</code> compound branch.
Hidden coupling concerns	Two-pass SVG contract (Boundary B-08). If extracted independently of <code>ClinicalEngine.process()</code> , the compound routing logic risks separation from the single-site routing logic.
Synchronization concerns	Two-pass SVG compound highlight is a load-bearing ordering contract (Architecture Risk Consolidation, Section 2).
Render/orchestration adjacency	Direct orchestration adjacency.
Extraction tier	DEFERRED — Wave 2 (after <code>ClinicalEngine.process()</code>)
Human review required	Y
Claude Opus escalation required	Y
Recommended extraction order	Wave 2, after <code>ClinicalEngine.process()</code>
Safe-to-extract status	DEFER TO WAVE 2+

CANDIDATE-025: `calcAI()` and `calcAIMulti()`

Field	Value
Candidate name	<code>calcAI()</code> , <code>calcAIMulti()</code>
Current location/context	Function declarations in main application file
Category	Pure calculation helper (deferred)
Purity confidence	REQUIRES INVESTIGATION (Wave 2+ scope)
Reasons for classification	Explicitly forbidden for Wave 1 (Playbook Section 3.4). <code>calcAI()</code> is called by <code>applyAIRec()</code> and <code>whatIfApply</code> directly, bypassing <code>ClinicalEngine.process()</code> . These call sites carry undocumented behavioral coupling. Extraction without resolving those call sites is out of scope. The documented semantic divergence between <code>calcAI()</code> and <code>ClinicalEngine.process()</code> for restorative conditions is a known existing condition that must not be accidentally unified (Rule 11.8).
Direct dependencies	Recommendation label maps (Tier 1), potentially the same sub-functions as <code>ClinicalEngine</code> — which is precisely the coupling risk.
Reverse dependencies (major callers)	<code>applyAIRec()</code> (forbidden), <code>whatIfApply</code> click handler (forbidden).
Hidden coupling concerns	The undocumented behavioral coupling with <code>applyAIRec()</code> and <code>whatIfApply</code> must be fully mapped before any extraction attempt. The divergence from <code>ClinicalEngine.process()</code> must be preserved, not unified.
Synchronization concerns	<code>applyAIRec()</code> and <code>whatIfApply</code> both directly mutate <code>S</code> — any extraction that touches <code>calcAI()</code> in proximity to these mutation sites risks altering mutation sequencing.
Render/orchestration adjacency	Adjacent to direct <code>S</code> mutation paths.
Extraction tier	DEFERRED — Wave 2+ (after call site coupling analysis)
Human review required	Y

Claude Opus escalation required	Y — E-03 conditions apply
Recommended extraction order	Wave 2+, after <code>ClinicalEngine.process()</code> sub-functions are validated and the <code>applyAIRec/whatIfApply</code> call sites are fully documented
Safe-to-extract status	DEFER TO WAVE 2+

Section 4: Permanently Forbidden Targets

The following targets are permanently forbidden for extraction as-is. Each prohibition is grounded in documented risks from the synchronization analysis, architecture risk consolidation, and DOM lifecycle audit. These are not deferred — they cannot be extracted in their current form under any wave without a fundamental architectural change that is out of scope.

FORBIDDEN-001: `render(S)`

Field	Value
Candidate name	<code>render(S)</code>
Category	Orchestration root
Purity confidence	FORBIDDEN
Reasons for classification	Root orchestrator of the entire render pipeline. Owns the second-pass SVG compound highlight overwrite directly. Calls all panel render functions. Forbidden at all waves until all dependencies are extracted (Playbook Section 3.1).
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-002: `renderMainPanels()`

Field	Value
Candidate name	<code>renderMainPanels()</code>
Category	DOM owner / orchestration
Purity confidence	FORBIDDEN
Reasons for classification	Carries <code>withErrorBoundary</code> , typewriter, material timer, SVG first-pass. If extracted without the second-pass overwrite from <code>render()</code> , compound SVG highlighting breaks permanently (Architecture Risk Consolidation, Section 2; DOM Lifecycle Audit, Section 9).
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-003: `setState()`

Field	Value
Candidate name	<code>setState()</code>
Category	State mutation / orchestration
Purity confidence	FORBIDDEN
Reasons for classification	State validator and diff tracker. Primary mutation entry point. Triggers save chain. Architecture Risk Consolidation marks this as EXTREME severity (S1).
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-004: `saveState()`

Field	Value
Candidate name	<code>saveState()</code>
Category	State mutation / persistence
Purity confidence	FORBIDDEN
Reasons for classification	Mutates persistence, owns <code>_saveStateTimer</code> , triggers <code>renderPatientList()</code> as a hidden side effect (Rule 11.4, Boundary B-04). The 200ms debounce chain is a load-bearing behavioral contract.
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-005: `switchPatient()`

Field	Value
Candidate name	<code>switchPatient()</code>
Category	State mutation / orchestration
Purity confidence	FORBIDDEN
Reasons for classification	Full S replacement, 300ms deferred render, cross-patient corruption window management, <code>showSkeleton()</code> ownership, <code>History.render()</code> orchestration. Architecture Risk Consolidation marks this as HIGH/CRITICAL (Boundary B-10).
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-006: `selectTx()`

--	--

Field	Value
Candidate name	<code>selectTx()</code>
Category	State mutation
Purity confidence	FORBIDDEN
Reasons for classification	Direct <code>S.tx</code> mutation plus render trigger. Contains early-exit branch for <code>S.multiTooth</code> that skips <code>History.add()</code> (Rule 11.10).
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-007: `applyAIRec()`

Field	Value
Candidate name	<code>applyAIRec()</code>
Category	State mutation
Purity confidence	FORBIDDEN
Reasons for classification	Direct <code>S.tx</code> mutation, bypasses <code>setState()</code> , calls <code>calcAI()</code> directly. The bypass is a documented undocumented behavioral coupling that must not be disturbed.
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-008: `toggleMultiSite()`

Field	Value

Candidate name	<code>toggleMultiSite()</code>
Category	State mutation / DOM owner
Purity confidence	FORBIDDEN
Reasons for classification	Direct <code>S.multiSite</code> mutation, DOM pre-wipe of <code>#comparisonTableBody</code> , render trigger. Architecture Risk Consolidation Section 10 marks Multi-Site DOM Resets as CRITICAL severity (S7).
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-009: `typewriterEffect()`

Field	Value
Candidate name	<code>typewriterEffect()</code>
Category	Timer owner / DOM owner
Purity confidence	FORBIDDEN
Reasons for classification	Registers <code>_typewriterTimer</code> (30ms <code>setInterval</code>), module-scoped singleton. DOM writer to <code>recText</code> . Timer is not in <code>CleanupRegistry</code> — managed only by its own next invocation. Extracting this would sever the cancellation-on-next-call pattern (Rule 11.1, Boundary B-05).
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-010: `buildAICardStructure()` / `updateAICard()` / `updateAICardMulti()`

Field	Value

Candidate name	<code>buildAICardStructure()</code> , <code>updateAICard()</code> , <code>updateAICardMulti()</code>
Category	DOM owner / structural rebuild
Purity confidence	FORBIDDEN
Reasons for classification	DOM owners with <code>dataset.built</code> flag contract. <code>buildAICardStructure</code> must precede <code>updateAICard</code> (ordering contract). <code>_typewriterTimer</code> registration in update path. The <code>dataset.built</code> flag is the sole determinant of structural vs. patch render — must not be moved (Rule 11.2, Architecture Risk Consolidation Section 2).
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-011: `renderComparisonTable()` / `lazyRenderComparisonTable()`

Field	Value
Candidate name	<code>renderComparisonTable()</code> , <code>lazyRenderComparisonTable()</code>
Category	DOM owner / observer registration
Purity confidence	FORBIDDEN
Reasons for classification	Observer registration (<code>_compTableObserver</code>), closure capture of <code>state</code> and <code>ai</code> , stale capture behavior is the documented existing behavior (Rule 11.5, Boundary B-07). Observer lifecycle is frozen.
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-012: `renderMaterial()`

Field	Value
--------------	--------------

Candidate name	<code>renderMaterial()</code>
Category	Timer owner / DOM owner
Purity confidence	FORBIDDEN
Reasons for classification	Owns <code>_matFadeTimer</code> (160ms <code>setTimeout</code>). DOM writer to <code>#matPrimary</code> and <code>#matAlt</code> . The 160ms window is a documented behavioral contract (Rule 11.6, Boundary B-05). Timer not in CleanupRegistry.
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-013: `renderTxCards()`

Field	Value
Candidate name	<code>renderTxCards()</code>
Category	DOM owner
Purity confidence	FORBIDDEN
Reasons for classification	Load-bearing ordering dependency with <code>updateCrownCardState()</code> . <code>updateCrownCardState()</code> reads <code>.classList</code> DOM state written by <code>renderTxCards()</code> in the same render cycle (Architecture Risk Consolidation Section 2).
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-014: `updateCrownCardState()`

Field	Value

Candidate name	<code>updateCrownCardState()</code>
Category	DOM-read precondition
Purity confidence	FORBIDDEN
Reasons for classification	Reads <code>.classList</code> state written by <code>renderTxCards()</code> . DOM-read precondition (Section 5.4). Not a pure function — depends on prior DOM mutation in the same render cycle.
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-015: `withErrorBoundary()`

Field	Value
Candidate name	<code>withErrorBoundary()</code>
Category	DOM owner / error recovery
Purity confidence	FORBIDDEN
Reasons for classification	Wraps DOM container IDs and performs <code>innerHTML</code> replacement on failure. The terminal error state (error card persists until force-rebuild) is a frozen behavioral contract (Rule 11.3, Boundary B-06). Not a pure utility.
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-016: `genCaseNum()`

Field	Value
Candidate	

name	<code>genCaseNum()</code>
Category	Impure function
Purity confidence	FORBIDDEN
Reasons for classification	Reads from <code>loadAllPatients()</code> which reads <code>localStorage</code> . Not pure — has an implicit external data dependency (Playbook Section 3.4). Additionally, the case number duplication collision (Architecture Risk Consolidation Section 9) makes this a data-integrity risk that must not be disturbed.
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-017: `History.add()` / `History.render()` / `History.save()` /

`History.load()`

Field	Value
Candidate name	<code>History.add()</code> , <code>History.render()</code> , <code>History.save()</code> , <code>History.load()</code>
Category	Persistence + DOM owner chain
Purity confidence	FORBIDDEN
Reasons for classification	Persistence mutation (<code>History.save()</code>), DOM owner (<code>History.render()</code> writes <code>#historyList</code>), <code>localStorage</code> reads (<code>History.load()</code>). History ID scope race condition documented (Architecture Risk Consolidation Section 8, DOM Lifecycle Audit Section 6). Chain must not be disturbed.
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-018: `updatePreview()` / `debouncedUpdatePreview()`

Field	Value
Candidate name	<code>updatePreview()</code> , <code>debouncedUpdatePreview()</code>
Category	Timer owner / partial render path
Purity confidence	FORBIDDEN
Reasons for classification	Timer owner (<code>_isPreviewMode</code> mutation), partial render path, directly mutates <code>ai._previewScore</code> onto the object returned from <code>ClinicalEngine.process()</code> (Section 5.9, Rule 11.8). The preview vs. apply race condition (Architecture Risk Consolidation Section 1) is a documented frozen behavioral contract.
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-019: `applyPreset()`

Field	Value
Candidate name	<code>applyPreset()</code>
Category	State mutation
Purity confidence	FORBIDDEN
Reasons for classification	<code>Object.assign(tempState, ...)</code> direct mutation, calls <code>updatePreview()</code> directly. Part of the preset vs. slider debounce race condition (Architecture Risk Consolidation Section 1).
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-020: `generateReport()`

Field	Value
Candidate name	<code>generateReport()</code>
Category	DOM owner / timer owner
Purity confidence	FORBIDDEN
Reasons for classification	Report button DOM mutation, orphaned 800ms timer (not in CleanupRegistry), URL allocation. Reads from <code>s</code> not <code>tempState</code> — this is a frozen behavioral contract even when preview mode is active (DOM Lifecycle Audit Section 7).
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-021: `buildEditForm()`

Field	Value
Candidate name	<code>buildEditForm()</code>
Category	DOM owner
Purity confidence	FORBIDDEN
Reasons for classification	Full <code>innerHTML</code> replacement of <code>#editFields</code> . DOM owner. Reads directly from current <code>S</code> values (DOM Lifecycle Audit Section 8).
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-022: `initClinicalNotes()`

Field	Value
Candidate name	<code>initClinicalNotes()</code>
Category	Listener binding / state mutation
Purity confidence	FORBIDDEN
Reasons for classification	Listener binding with <code>_notesInputBound</code> guard, <code>s.notes</code> mutation. The notes autosave silent abort (Rule 11.7) is a frozen behavioral contract — extraction that disturbs <code>_notePatient</code> capture would alter this contract.
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-023: `updateSliderPositions()`

Field	Value
Candidate name	<code>updateSliderPositions()</code>
Category	DOM owner
Purity confidence	FORBIDDEN
Reasons for classification	DOM writes to slider elements from <code>tempState / s</code> . Reads state and writes DOM — not a pure function.
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-024: `cancelEdit()` / `exitEdit()` / `saveEdit()`

Field	Value
Candidate name	<code>cancelEdit()</code> , <code>exitEdit()</code> , <code>saveEdit()</code>

Category	DOM mutation / state mutation
Purity confidence	FORBIDDEN
Reasons for classification	DOM and state mutation. <code>saveEdit()</code> calls <code>setState()</code> and triggers render chain. <code>exitEdit()</code> mutates <code>style.display</code> on DOM elements. None are pure.
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-025: `showSkeleton()`

Field	Value
Candidate name	<code>showSkeleton()</code>
Category	DOM owner
Purity confidence	FORBIDDEN
Reasons for classification	DOM owner that clears <code>dataset.built</code> implicitly by replacing <code>#aiCardBody</code> innerHTML. The <code>dataset.built</code> flag integrity contract (Rule 11.2) must not be disturbed. <code>showSkeleton()</code> is part of the <code>switchPatient()</code> frozen orchestration sequence.
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-026: `whatIfApply` click handler

Field	Value
Candidate name	<code>whatIfApply</code> click handler

Category	State mutation
Purity confidence	FORBIDDEN
Reasons for classification	<code>Object.assign(S, ...)</code> bypasses <code>setState()</code> . The <code>tempState</code> selective promotion contract (Rule 11.9, Architecture Risk Consolidation Section 9, Boundary B-02) requires the field mapping to remain exactly in-place.
Safe-to-extract status	NEVER EXTRACT AS-IS

FORBIDDEN-027: All module-scoped timer singletons

Field	Value
Candidate name	<code>_typewriterTimer</code> , <code>_matFadeTimer</code> , <code>_saveStateTimer</code> , <code>_notesDebounce</code> , <code>_compTableObserver</code> , <code>_lastSimAI</code> , <code>_isPreviewMode</code> , <code>_notePatient</code> , <code>_notesInputBound</code>
Category	Async runtime singletons
Purity confidence	FORBIDDEN
Reasons for classification	All module-scoped mutable singletons. Any extraction candidate that reads or writes these is automatically forbidden (Playbook Section 5.1, Boundary B-05, Boundary B-07). These singletons remain exclusively in the main application file under all Wave 1 activity.
Safe-to-extract status	NEVER EXTRACT AS-IS

Registry Summary

Section	Count
Approved Wave 1 Candidates	18 (CANDIDATE-001 through CANDIDATE-018)

Conditional Candidates Requiring Investigation	4 (CANDIDATE-019 through CANDIDATE-022)
Deferred Wave 2+ Candidates	3 (CANDIDATE-023 through CANDIDATE-025)
Permanently Forbidden Targets	27 entries (FORBIDDEN-001 through FORBIDDEN-027)

Approved Candidates by Tier

Tier	Candidates	Notes
Tier 1	CANDIDATE-001 through CANDIDATE-008	Constants and lookup tables
Tier 2	CANDIDATE-009 through CANDIDATE-013	String and number formatters
Tier 3	CANDIDATE-014 (approved), CANDIDATE-021 (conditional)	Validators
Tier 4	CANDIDATE-015 through CANDIDATE-018 (approved), CANDIDATE-019 and CANDIDATE-020 (conditional)	Pure clinical calculation helpers
Tier 5	CANDIDATE-022 (conditional)	Conditional extractions

Human Review Required

CANDIDATE-007, CANDIDATE-015, CANDIDATE-016, CANDIDATE-017, CANDIDATE-018, CANDIDATE-019, CANDIDATE-020, CANDIDATE-021, CANDIDATE-022

Claude Opus Escalation Conditional

CANDIDATE-007 (if transient flag boundary is ambiguous), CANDIDATE-016 (if ClinicalEngine routing branch boundary is ambiguous), CANDIDATE-018 (if calcAI vs. ClinicalEngine divergence cannot be traced), CANDIDATE-019 (if ai object mutation is ambiguous), CANDIDATE-020 (if DOM-read dependency discovered), CANDIDATE-021 (if side-channel write discovered), CANDIDATE-023, CANDIDATE-024, CANDIDATE-025

Registry status: Governance intelligence only. No refactoring, redesign, optimization, or architectural change is contained herein or implied. All classifications are derived from documented system behavior in the synchronization analysis, render trigger chain analysis, architecture risk consolidation, and DOM lifecycle audit. Extraction eligibility is subject to pre-extraction checks per Wave 1 Execution Playbook Section 4 before any extraction begins.